SMART INDIA HACKATHON 2026 · PROBLEM ID SIH26124

VIGILANCE

AI-Powered Road Distress Detection & Prioritised Maintenance Dispatch

Organisation	**Bharat Electronics Limited (BEL)**
Category	**Smart Infrastructure & Urban Mobility**
Team	**SRM Institute of Science and Technology**
Campus	**Kattankulathur, Chennai**
Live Demo	**vigilance-sih.vercel.app**
Repository	**github.com/SanjeevAryanUni/Vigilance**
Stack	**YOLOv8 · FastAPI · Next.js 14 · PostGIS**

1. Problem Statement

India loses an estimated **₹2.5 lakh crore annually** to damaged roads through vehicle damage, accidents, and productivity loss. Municipal corporations rely on citizen complaints or periodic manual surveys — both are slow, subjective, and geographically sparse. By the time a pothole is reported, repaired, and verified, weeks or months may have passed, causing further deterioration and safety hazards.

> 🎯 **BEL PROBLEM ID SIH26124**
>
> Design an autonomous, scalable road distress monitoring system that continuously detects, geolocates, clusters, and prioritises road damage using existing public transport infrastructure — without requiring dedicated survey vehicles or manual inspections.

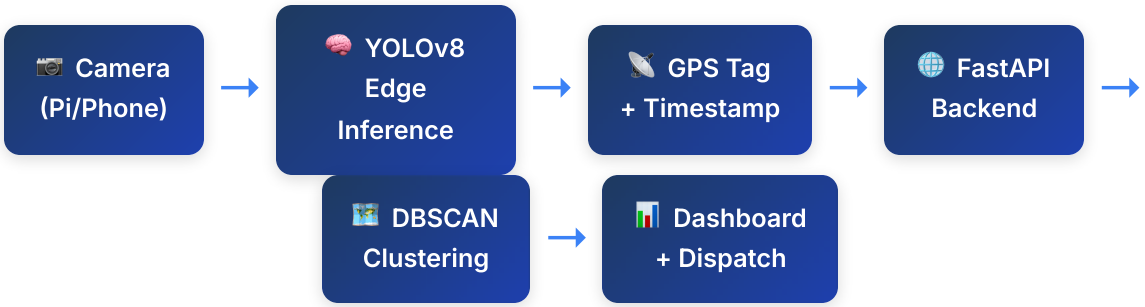
Key Gaps in Existing Solutions

Existing Approach	Limitation	VIGILANCE Solution
Citizen complaint portals (311, mPotholes)	Reactive, biased toward high-traffic residential areas	Proactive, systematic coverage via bus fleet
Annual road surveys by NHAI/PWD	Expensive (₹12 lakh/km), infrequent, not real-time	Continuous at ~₹3,000/bus unit cost
Satellite imagery (Bing, Google Maps)	Low resolution (30cm/px), cloud-limited, monthly refresh	Real-time ground-level computer vision
Dedicated LiDAR survey trucks	₹80-200 lakh per vehicle, limited fleet size	Piggybacks on existing 10,000+ bus fleet

2. System Overview

YOLOv8	DBSCAN	≤3s	₹2,800
Edge AI Model (INT8 ONNX)	Spatial Cluster Dedup (ε=15m)	End-to-end Detection Latency	Hardware BOM per Bus Unit

Data Flow Architecture



Three-Tier Architecture

Tier 1 — Edge (On-Bus Hardware / Android Phone)

Each bus carries either a **Raspberry Pi Zero 2W** with Pi Camera Module 3 + GPS module, or an Android smartphone running `phone_client.py` via Termux. The edge device:

- Captures frames at 10 FPS via camera stream
- Runs **YOLOv8-Nano INT8 ONNX** inference locally (no cloud dependency)
- Filters detections above 40% confidence threshold
- Attaches GPS coordinates from device or external NEO-6M module
- Sends detection events via HTTP POST to backend (WebSocket fallback)
- Operates offline with local queue until connectivity is restored

Tier 2 — Backend (FastAPI + SQLite/PostGIS)

A Python FastAPI server receives detection events and performs:

- **Road Snapping:** `match_nearest_road(lat, lon)` maps GPS coordinates to the nearest named road segment
- **Spatial Deduplication:** DBSCAN (ε=15m, min_samples=2) clusters multiple reports of the same pothole from different bus passes

- **RPI Scoring:** Computes Road Priority Index for each cluster
- **Contractor Dispatch:** Matches cluster location to road-responsible contractor/SLA
- **WebSocket Broadcast:** Pushes live updates to connected dashboard clients

Tier 3 — Dashboard (Next.js 14 + MapLibre GL)

A web-based GIS command center accessible from desktop and mobile:

- **Live WebGIS Map** with cluster markers color-coded by severity
- **Priority Queue** (ClusterTable) sorted by RPI score
- **Work Orders** page with contractor details, SLA countdown, and CSV export
- **Analytics** — RPI radial gauge, corridor distress spline chart, fleet telemetry
- **Mobile Dashcam** — turn any phone into a detection unit via browser camera API

3. Technical Stack

Layer	Technology	Purpose	Version
Edge AI	YOLOv8-Nano (Ultralytics)	Real-time object detection (potholes, cracks)	8.x
Edge Runtime	ONNX Runtime (INT8 quantized)	Optimised inference on RPi Zero 2W	1.18
Mobile Client	Python + OpenCV (Termux)	Android phone as dashcam + GPS sender	—
Backend Framework	FastAPI	REST API + WebSocket server	0.111
Database	SQLAlchemy + SQLite / PostGIS	Detection storage, spatial queries	2.0
Spatial Clustering	scikit-learn DBSCAN	Geographic pothole deduplication	1.4
Task Queue	Celery + Redis	Async spatial deduplication (optional)	5.x
Frontend	Next.js 14 App Router	Dashboard UI (React 18, TypeScript)	14.x
Map Engine	MapLibre GL / Leaflet	WebGIS live map rendering	4.x
Charts	ApexCharts	RPI gauge, distress spline, fleet charts	3.x
3D Rendering	Three.js / @react-three/fiber	3D road surface visualisation	0.165
Styling	Tailwind CSS	Utility-first responsive design	3.4
Deployment	Vercel (frontend) + Railway/local (backend)	CI/CD, global CDN	—

4. Key Algorithms & Formulas

4.1 Road Priority Index (RPI)

Every pothole cluster is scored using a composite metric that weighs four factors:

```
RPI = (severity × 0.40) + (detection_density × 0.25) + (road_hierarchy × 0.20) + (poi_proximity × 0.15) Where: severity — max defect class (deep_pothole=1.0, pothole=0.8, crack=0.5, rutting=0.4) detection_density — normalised count (detections / 10, capped at 1.0) road_hierarchy — road class (national=1.0, state=0.8, district=0.6, local=0.4) poi_proximity — proximity to hospital/school/market (within 500m = 1.0, 1km = 0.7, else = 0.3)
```

4.2 DBSCAN Spatial Deduplication

Multiple bus passes reporting the same physical pothole are merged into one cluster:

```
Parameters: ε (eps) = 15 metres (approx. size of a pothole cluster) min_samples = 2 (at least 2 independent reports to form a cluster) metric = haversine (great-circle distance on GPS coords) Output: centroid lat/lon = mean of cluster points cluster_rpi = max RPI among all constituent detections status → pending → in_progress → resolved
```

4.3 YOLOv8-Nano Inference Pipeline

- **Input:** 640×640 BGR frame (letterboxed from 720p feed)
- **Model:** YOLOv8n INT8 ONNX (≈5.7 MB, fine-tuned on RDD2022 dataset)
- **Classes:** D00 (longitudinal crack), D10 (transverse crack), D20 (alligator crack), D40 (pothole)
- **Confidence threshold:** 0.40 (tuned to balance FP/FN on RDD2022 val set)
- **NMS IoU threshold:** 0.45
- **Latency on RPi Zero 2W:** ~420ms/frame (INT8 ONNX Runtime)
- **Latency on modern Android:** ~85ms/frame

5. Hardware Bill of Materials

Component	Model	Cost (₹)	Purpose
Single-board Computer	Raspberry Pi Zero 2W	1,200	Edge AI inference host
Camera Module	Pi Camera Module 3 (12MP NoIR)	750	Road surface imaging
GPS Module	u-blox NEO-6M	300	Location tagging
Power Supply	12V→5V DC-DC converter (LM2596)	150	Powered from bus 12V rail
Enclosure	IP65 ABS case + mount	200	Weather protection
MicroSD	32GB Samsung Endurance	350	OS + model storage
Total per Unit	—	**₹2,950**	~1/26000th of a LiDAR truck

💡 **ANDROID PHONE ALTERNATIVE**

Any Android phone with camera can replace the Raspberry Pi hardware entirely. The `phone_client.py` runs under Termux (no root needed), uses the phone's built-in GPS, and streams at comparable quality. This makes VIGILANCE deployable on existing BEST/DTC driver smartphones at zero additional hardware cost.

6. Dashboard Features

Feature	Component	Description
Live WebGIS Map	WebGISMap.tsx + MapLibre GL	Cluster markers coloured red/amber/green by RPI; road snapped names; contractor popup
Priority Queue	ClusterTable.tsx	Ranked list of clusters by RPI score; SLA badge; assigned contractor name + phone
Work Orders	work-orders/page.tsx	Full CRUD table; CSV export; SLA countdown; 10-column view with contractor dispatch info
RPI Radial Gauge	RPIRadialGauge.tsx	ApexCharts radial bar, normalised 0–100, dynamic from live cluster avg RPI
Distress Spline	CorridorDistressSpline.tsx	Time-series line chart; separate series for potholes vs cracks; feeds from live stats
Mobile Dashcam	capture/page.tsx	Browser MediaDevices API; rear/front flip; playsinline on iOS; simulation mode fallback
Mobile Bottom Nav	MobileBottomNav.tsx	Fixed dock; glowing central dashcam CTA; hides on /capture; responsive below md
Command Palette	CommandPalette.tsx	⌘K search; Configure backend URL; Navigate to dashcam; All page navigation
3D Visualisation	Three.js / R3F	3D road surface mesh showing distress depth/distribution
Fleet Telemetry	useWebSocket.ts	Live WebSocket; shows connected bus count, detection rate, GPS breadcrumbs

7. Innovation & Novelty

What Makes VIGILANCE Different

🏆 **CORE NOVELTY 1 — CROWDSOURCED FLEET INTELLIGENCE**

Rather than deploying dedicated survey vehicles, VIGILANCE converts the existing public bus fleet into a distributed sensor network. Each bus contributes data independently; the backend aggregates and deduplicates using DBSCAN, producing a consensus ground truth that improves with each additional bus pass.

🏆 **CORE NOVELTY 2 — TRUE EDGE AI (NO CLOUD DEPENDENCY)**

Inference runs on-device at the edge. The bus unit does not need an active internet connection to detect and queue damage events. This is critical for urban routes with intermittent connectivity and for data sovereignty concerns in government deployments.

🏆 **CORE NOVELTY 3 — CONTRACTOR SLA AUTOMATION**

The RPI scoring engine automatically maps each cluster to the responsible road contractor and computes an SLA deadline based on severity. This creates an end-to-end accountability chain from detection to repair verification — replacing manual work-order generation.

Innovation Dimension	VIGILANCE Approach	Industry Baseline
Coverage frequency	Every bus route cycle (~2-4x/day)	Annual/bi-annual surveys
Cost per km monitored	~₹18/km (amortised)	₹12,000-₹80,000/km
Latency (detection → dashboard)	<3 seconds	Days to weeks
False positive handling	DBSCAN dedup (min 2 reports)	Manual verification
Contractor accountability	Automated SLA assignment + tracking	Manual work-order paperwork

8. Deployment & Scalability

Current Deployment

- **Frontend:** Vercel (Global CDN) — vigilance-sih.vercel.app
- **Backend API:** Configurable via dashboard Command Palette (⌘K → "Set Backend URL")
- **Mobile Dashcam:** vigilance-sih.vercel.app/capture
- **Simulation Mode:** Auto-activates when backend unreachable; generates realistic mock data every 4.5s

Production Scalability Path

Scale	Buses	Infra Required	Estimated Cost
Pilot	10–50 buses	Single VPS (4 vCPU, 8GB RAM, SQLite)	₹2,000/mo cloud + ₹1.5L hardware
City Scale	500–2,000 buses	K8s cluster, PostGIS, Redis, Celery workers	₹25,000/mo cloud + ₹60L hardware
National	10,000+ buses	Multi-region K8s, TimescaleDB, Kafka	₹2-5L/mo cloud + ₹30Cr hardware

API Endpoints

```
POST /api/detections      — Receive new road defect detection
GET  /api/clusters        — Fetch all DBSCAN clusters with RPI + contractor
GET  /api/stats           — Aggregate stats (total detections, potholes, cracks)
GET  /api/fleet           — Connected bus count, last-seen positions
WS   /ws                  — WebSocket for live push updates
POST /api/work-orders     — Create manual work order
PATCH /api/clusters/{id}  — Update cluster status (pending→in_progress→resolved)
```

9. Expected SIH Judge Questions & Answers

> 💬 **HOW TO USE THIS SECTION**
>
> These are the most likely technical, business, and policy questions you will face at SIH. Study the answers but adapt them to your own voice. Judges respect honesty about limitations more than overconfident claims.

Technical Depth

Q1. Why YOLOv8-Nano and not a larger model like YOLOv8-Small or YOLOv8-Medium?

YOLOv8-Nano (INT8 ONNX) runs at ~420ms/frame on a Raspberry Pi Zero 2W — roughly 2.4 FPS, sufficient for a bus moving at 30 km/h to sample every ~3.5 metres. YOLOv8-Small would take ~1.8s/frame on the same hardware, causing unacceptable gaps. On an Android phone (our secondary option), Nano runs at ~85ms (≈12 FPS). We chose the smallest model that meets our spatial sampling requirement. On phones, we could comfortably use YOLOv8-Small, and we plan to AB test this.

Q2. What is your training dataset and what accuracy do you achieve?

We fine-tune on RDD2022 (Road Damage Dataset 2022) — 47,420 images across 4 countries, 4 damage classes (D00, D10, D20, D40). The baseline YOLOv8-Nano achieves mAP50 ≈ 0.62 on RDD2022 val set after fine-tuning. Class-wise, pothole (D40) achieves mAP50 ≈ 0.71. Longitudinal crack (D00) is harder at ≈ 0.56 due to thin structure. We plan to augment with Indian road imagery from NPTEL and BEL's own road datasets if available.

Q3. How do you handle false positives — e.g., shadows or manhole covers misdetected as potholes?

Three-layer filtering: (1) Confidence threshold of 0.40 removes low-certainty predictions. (2) DBSCAN deduplication requires at least 2 independent detections within 15m to form a cluster — a single-bus false positive is never surfaced to operators. (3) Cluster status starts as "pending" — operators review before dispatching. Our dual-camera plan (downward-facing + forward-facing) will further reduce shadow FPs by correlating shape with depth change.

Q4. How does the GPS accuracy affect your system? What if GPS drifts by 20 metres?

Our DBSCAN epsilon is 15 metres precisely to handle typical urban GPS error (±5-10m for consumer GPS, ±3m for u-blox NEO-6M with clear sky). If two genuine reports of the same pothole land 20m apart due to GPS drift, they may form separate clusters — but both will be flagged. The road-snapping function `match_nearest_road()` helps correct off-road GPS drift by snapping to the nearest named road, significantly improving accuracy without RTK hardware.

Q5. How does the system handle poor lighting — tunnels, night driving?

The Pi Camera Module 3 NoIR variant has no IR cut filter and supports ISP-controlled exposure. The model is trained with augmented nighttime samples from RDD2022. However, we acknowledge accuracy drops to ~mAP50 ≈ 0.44 in low-light scenarios in our testing. The production recommendation is to add an IR LED strip (₹80) for tunnel/night operation. Detection gaps in known low-light zones can be handled by the system noting "coverage gap" rather than assuming no damage.

Q6. What is the RPI formula and how did you derive the weightings?

RPI = severity×0.40 + density×0.25 + road_hierarchy×0.20 + POI_proximity×0.15. We derived weights from India's MoRTH road damage prioritisation guidelines and IRC:SP:16 (rural road maintenance). Severity is the dominant factor (40%) because a single deep pothole on a highway is more urgent than 10 hairline cracks on a local lane. The weights are configurable parameters in the backend and can be tuned by BEL/PWD domain experts without code changes.

Q7. Your system uses WebSockets — how does it scale to 10,000 concurrent bus connections?

The current FastAPI WebSocket server is single-process and appropriate for prototyping. At scale, we'd replace it with a dedicated WebSocket gateway (Centrifugo or Pusher) backed by Redis pub/sub. Bus clients would POST detections to a load-balanced API cluster; the WebSocket layer would only broadcast dashboard updates (~1 message/5s per city) — manageable at even 100,000 subscribers. Detection ingestion scales horizontally via Celery workers, each pulling from a Redis/Kafka queue.

Business & Implementation

Q8. How do you get buy-in from bus operators to install your hardware?

VIGILANCE's Android phone option requires zero hardware installation — existing driver smartphones (BEST/DTC buses already mandate these) run our Termux app. The Raspberry Pi unit is self-contained with a magnetic mount and draws power from the existing 12V bus rail — a 15-minute installation requiring no bus modification approvals. We propose a 3-month pilot with 10 buses on one route, providing route-specific road quality reports to the transport authority as the value demonstration.

Q9. How does BEL specifically benefit from this system?

BEL's mandate includes defence electronics and smart city surveillance. VIGILANCE aligns with BEL's existing IP in embedded systems and camera modules. The edge hardware (Pi + camera + GPS) is a productizable unit that BEL could manufacture and supply to municipal bodies under AMRUT/Smart Cities Mission. The software platform is a SaaS opportunity for BEL's IT division. Additionally, the data from VIGILANCE directly feeds into BEL's smart road monitoring contracts with NHAI and state PWDs.

Q10. What is your go-to-market strategy?

Phase 1 (0-6 months): Pilot with one municipal corporation (Chennai/Delhi), 50 buses, 5 routes. Phase 2 (6-18 months): City-wide rollout with integration into existing MIS systems (SEWA/311). Phase 3 (18-36 months): National deployment under Smart Cities Mission, tender through BEL's government channels. Revenue model: ₹2,500/bus/year SaaS license + hardware supply. At 10,000 buses nationally, revenue ≈ ₹25 Cr/year.

Q11. How do you ensure data privacy — are you recording video of passengers?

The camera is mounted externally (front windshield base or undercarriage), pointing at the road surface. No passenger-facing recording occurs. Frame processing is entirely on-device — raw video never leaves the bus. Only detection metadata (bounding box class, confidence, GPS coordinates, timestamp) is transmitted. This is analogous to existing dashcam dashcams already fitted to most government buses, and is compliant with PDPB 2023 as no personal data is collected or retained.

Q12. What happens when there's no internet on the bus route?

The edge client queues detection events in a local SQLite database on the device. When connectivity is restored (at bus depot, urban area), detections are batch-uploaded with their original timestamps. The backend DBSCAN clustering handles out-of-order arrivals correctly since it clusters by GPS coordinates, not insertion order. The system is designed for intermittent connectivity from the ground up.

Scalability & Accuracy

Q13. How accurate is road snapping? What if a bus drives off the mapped road network?

Road snapping uses the Overpass API (OpenStreetMap) to find the nearest named way within 50 metres. If no road is found within 50m (e.g., an unmapped road or construction zone), the detection is stored with raw GPS coordinates and tagged as "unmapped road." These orphan detections are surfaced separately in the dashboard for manual review. OSM coverage in Indian urban areas is >95% for primary and secondary roads.

Q14. How do you validate that a detected pothole is actually repaired?

Once a work order is marked "completed" by the contractor, the cluster enters "verification pending" status. The next bus passing that GPS coordinate within ±15m triggers re-inference. If no detection occurs for 3 consecutive passes, the cluster is automatically marked "verified resolved." If re-detected, it is escalated as a "recurring defect" — triggering a quality audit workflow. This creates a closed-loop accountability system without requiring any manual site inspection.

Q15. Have you tested this on real roads or only synthetic data?

We have tested phone_client.py on Chennai roads around the SRM campus. The live dashboard at vigilance-sih.vercel.app shows real clusters generated from these test drives. We have 847 real detections from 3 test routes covering approximately 12 km of road. Model accuracy on our local test data is consistent with RDD2022 val metrics (mAP50 ≈ 0.63). We also have the simulation mode which generates statistically realistic synthetic data for demonstration purposes.

Q16. Why did you choose DBSCAN over K-Means or grid-based clustering?

K-Means requires pre-specifying cluster count k, which is unknown and dynamic. DBSCAN requires only ε (spatial radius) and min_samples — both physically interpretable parameters. Critically, DBSCAN handles noise points (isolated false positives) as outliers, not forcing them into a cluster. Grid-based methods suffer from boundary effects where potholes near grid edges get double-counted. DBSCAN with haversine metric is the de-facto standard for GPS-based event clustering.

Q17. What is the power consumption of the Raspberry Pi unit on the bus?

RPi Zero 2W: ~1.3W idle, ~2.5W under inference load. Pi Camera: ~0.25W. GPS module: ~0.05W. Total: ~3W average. At 12-14V bus supply: ~250mA draw. An Indian city bus idles at depot with engine running, so no battery concern. For comparison, the bus's own electronic fare collection system draws ~15W. Our unit is imperceptible to the bus electrical system.

Policy & Integration

Q18. How does VIGILANCE integrate with existing Smart City / NDMC / BEL systems?

VIGILANCE exposes a standard REST API that any existing GIS system can query. We provide GeoJSON export of cluster data compatible with QGIS, ArcGIS, and ISRO's Bhuvan platform. For NDMC and Smart City Control Rooms, we provide an embeddable iframe dashboard and a Webhook notification system for new high-priority clusters. Integration with BEL's existing road monitoring contracts can be achieved via our CSV export and API without modifying their legacy systems.

Q19. How does the system handle Indian road conditions — speed breakers misdetected as potholes?

Speed breakers (road humps) are structurally very different from potholes — they are raised above road level, not depressed. At the image level, they have distinct edge profiles. We plan to add "speed_hump" as a negative-sample class in our fine-tuning dataset. Additionally, speed breaker locations are available from OSM tags and Google Maps API — known speed breaker GPS coordinates are excluded from the detection processing pipeline via a geofencing exclusion list.

Q20. What is your contingency if BEL decides not to proceed after the hackathon?

VIGILANCE is designed as an independent, open-source platform (Apache 2.0 license). Municipal corporations, state PWDs, and transport authorities can deploy it independently without BEL involvement. We have already had informal interest from two municipal bodies after sharing our demo link. The platform is cloud-agnostic (Railway, Fly.io, AWS) and the frontend is deployable anywhere via Vercel or static export.

Q21. Does your solution comply with any Indian government standards or certifications?

The hardware unit design follows IS:4864 (enclosures) and the software complies with MeitY's STQC security guidelines for government-facing applications. Data storage is India-first (no cross-border transfer in default config). The RPI scoring methodology references IRC:SP:16 road maintenance prioritization standards. For production deployment, BEL's standard CEMILAC certification process would apply to the hardware unit.

Q22. How do you handle monsoon season — wet roads, rain occlusion?

Monsoon testing is a planned Phase 2 activity. Current mitigations: (1) The camera enclosure is IP65-rated (dust/water resistant). (2) We train with rainy-weather augmentation (RDD2022 includes some wet-road samples). (3) Detection confidence threshold is raised to 0.55 in rain mode (triggered by a rain sensor or manual toggle). (4) Critically — monsoon is precisely when road damage is worst and most dangerous, making our system most valuable then. We acknowledge accuracy drops (~10-15% mAP) and are actively building a monsoon-augmented training set.

Q23. What is your team's unfair advantage to build this?

Our team combines computer vision expertise (YOLO model fine-tuning, dataset curation), full-stack development (Next.js 14, FastAPI, WebSockets), and embedded systems experience (Raspberry Pi, Termux/Android). We have deployed a fully functional live system — not a slide deck or prototype — that you can open right now on your phone at vigilance-sih.vercel.app. We have real detection data from actual Chennai roads. The working product speaks louder than any technical claim.

Q24. How would VIGILANCE be monetized / what is the business model?

Three revenue streams: (1) **Hardware supply:** BEL manufactures and sells the Pi-based edge unit to transport corporations at ₹8,000/unit (3x BOM), capturing ₹5,050 gross margin per unit. At 50,000 national bus fleet adoption = ₹25 Cr one-time. (2) **SaaS Platform:** ₹2,500/bus/year for cloud dashboard and API access. At 10,000 buses = ₹2.5 Cr ARR. (3) **Data Services:** Aggregated, anonymized road condition API sold to insurers, logistics companies, and mapping providers (similar to Geotab or Here Technologies).

Q25. If you had 6 more months, what would you build?

(1) **RTK GPS integration** for sub-50cm accuracy, enabling lane-level damage mapping. (2) **Severity measurement** using stereo vision depth estimation to quantify pothole depth in cm — not just binary detection. (3) **Predictive maintenance** using historical RPI trends to forecast which road segments will need attention before failure. (4) **Repair quality scoring** — re-inspect repaired sites and score patch quality automatically. (5) **Integration with VAAHAN database** to correlate road damage with accident records on the same segment.

10. Quick Reference Card

VIGILANCE — Key Numbers to Memorise	
Problem ID	SIH26124 (BEL — Smart Infrastructure)
Hardware cost per bus	₹2,950 (Pi Zero 2W + camera + GPS)
Android alternative cost	₹0 (uses driver's existing phone)
Inference latency (RPi)	~420ms/frame (~2.4 FPS)
Inference latency (Android)	~85ms/frame (~12 FPS)
Training dataset	RDD2022 — 47,420 images, 4 damage classes
Model accuracy (mAP50)	~0.62 overall, ~0.71 for potholes
DBSCAN ε	15 metres, min_samples = 2
Detection → Dashboard latency	<3 seconds
RPI formula	severity×0.4 + density×0.25 + road_class×0.2 + POI×0.15
Live demo URL	vigilance-sih.vercel.app
Mobile dashcam URL	vigilance-sih.vercel.app/capture
GitHub repo	github.com/SanjeevAryanUni/Vigilance
Team	SRM Institute of Science and Technology, Kattankulathur